

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood

System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
 - For a media platform that stores large amounts of videos, images, and audio files, HDFS can be used as the main distributed storage system. The design must maintain data availability above 99.9%, keep the replication factor at a maximum of 3, and control storage costs.
 - The HDFS cluster can use a **replication factor of 3** for important media data. Each block is stored on different DataNodes so that failure of one or even two nodes does not make the data unavailable. The NameNode should be configured with **High Availability (HA)** using an Active and Standby NameNode.
 - To reduce storage cost, older and less frequently accessed media can be moved to lower-cost storage or compressed. Data can also be organized using suitable block sizes to reduce unnecessary metadata overhead. Monitoring and automatic rebalancing can help maintain efficient storage utilization.
 - Thus, the design provides high availability through replication and NameNode HA while keeping the replication factor within the limit and controlling storage growth through compression and storage-tier management.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
 - The MapReduce workflow must process log data within a **20-minute batch window** while using a limited number of cluster nodes. The job should therefore be divided into parallel Map and Reduce tasks.
 - During the **Map phase**, input log files are divided into blocks and distributed among the available nodes. Each Mapper processes its assigned log records and extracts useful information such as IP address, user activity, error type, timestamp, or URL.
 - The intermediate results are then shuffled and grouped based on the required key. During the **Reduce phase**, Reducers aggregate the grouped data, such as counting requests, calculating error frequencies, or identifying frequently accessed pages.
 - Since the number of nodes is limited, the number of Mapper and Reducer tasks should be optimized according to the available CPU, memory, and input data size. Combiners can be used to perform local aggregation and reduce network traffic during the shuffle phase.
 - This parallel task distribution allows the log-processing job to complete within the 20-minute batch window without requiring a large cluster.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

- A YARN-managed cluster may need to run ETL, reporting, and machine learning jobs simultaneously. A suitable scheduler policy should provide fairness while giving higher priority to ETL jobs.
- A **Capacity Scheduler** can be used to divide cluster resources into different queues. For example, an ETL queue can receive higher capacity and priority, while reporting and machine learning jobs receive appropriate resources based on their requirements.
- The ETL queue can be given higher priority because ETL jobs prepare data required by other applications. Reporting jobs can receive medium priority, while long-running machine learning jobs can use the remaining resources without completely blocking other workloads.
- YARN's resource management and monitoring mechanisms can detect node failures. If a NodeManager fails, YARN can reallocate resources and restart failed application tasks on healthy nodes. ApplicationMaster recovery can also help applications continue after failures.
- Therefore, the scheduler provides **fair resource sharing, ETL priority, and fault recovery** while preventing one workload from consuming all cluster resources.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

- An enterprise may have data stored in **SAN, NAS, and operational databases**. A centralized Hadoop-based data platform can be designed to ingest all these sources while minimizing duplication and maintaining reliable backups.
- **Apache NiFi** can be used as the data ingestion layer. NiFi can collect files from NAS and SAN locations and extract data from operational databases. For database ingestion, tools such as **Apache Sqoop or JDBC-based pipelines** can be used depending on the environment.
- The collected data can be stored in HDFS or a suitable Hadoop-compatible storage layer. A **raw/landing zone** can preserve the original data, while processed and cleaned data can be stored separately. Metadata and data lineage should be maintained to identify the source of every dataset.
- To reduce duplication, unique identifiers, checksums, incremental ingestion, and change-data capture techniques can be used. Important datasets should have appropriate replication and backup policies.
- This architecture provides centralized ingestion, reliable backup, better data tracking, and reduced unnecessary duplication.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

- An end-to-end Hadoop solution can integrate **Apache HDFS, YARN, MapReduce, Apache NiFi, Hive, and security mechanisms** to process enterprise data.
- First, Apache NiFi collects data from databases, files, applications, and external systems. Secure protocols such as **SFTP, HTTPS, or encrypted connections** can be used during data movement. NiFi then transfers the data into HDFS.
- HDFS provides distributed storage with replication and **NameNode High Availability**. A replication factor of up to 3 can be used to protect data from node failures. YARN manages computing resources and schedules different applications according to their priority and resource requirements.
- MapReduce or other Hadoop processing engines can process large datasets, while Hive can provide SQL-based querying and reporting. ETL pipelines can clean and transform the incoming data before it is stored in analytical datasets.
- For security, authentication, authorization, encryption, and access control should be implemented. Regular backups, monitoring, and failure recovery mechanisms should also be included.
- Thus, the proposed Hadoop ecosystem provides **secure data movement, highly available storage, efficient resource management, ETL integration, and scalable data processing**, satisfying the major constraints of an enterprise big-data environment.